

ADOSX Junior Full-Stack Assignment

README

A Django + React reconciliation application that imports two system exports, identifies disagreements, enforces tenant boundaries, and displays the results through a simple filterable and sortable web interface.

1. Project Overview

The application compares records from System A and System B. The source data is intentionally dirty, so the importer is designed to preserve rows and handle missing or malformed values. Locations provide the organization (tenant) mapping, and reconciliation results are restricted to the appropriate organization.

2. Technology Stack

Layer	Technology
Backend	Python, Django, Django REST Framework
Frontend	React, Vite
Database	SQLite for local development / Django database models
API	REST endpoint for reconciliation disagreements
Frontend styling	Plain functional React UI
Data source	system_a.csv, system_b.csv, locations.csv

3. What I Built

- Imported System A, System B, and location data into Django models.
- Implemented reconciliation logic to detect missing System B entries, missing System A records, duplicate System B entries, and value mismatches.
- Used locations.csv to associate records with organizations and prevent cross-tenant visibility.
- Created a REST API endpoint for disagreement results.
- Built a React frontend that displays disagreements in a table.
- Added filtering by disagreement reason.
- Added sorting by value.
- Added automated tests for the core comparison logic, including tenant isolation.

4. Main Reconciliation Cases

Reason	Meaning
missing_in_system_b	A System A record has no corresponding System B entry.
missing_in_system_a	A System B entry references a System A record that does not exist.
duplicate_in_system_b	The same System A record is represented more than once in System B.
value_mismatch	System A and System B contain different values for the same record.

5. Project Structure

```
adosx-fullstack/  
  ■■■ backend/
```

```
■ ■■■ manage.py
■ ■■■ reconciliation/
■ ■ ■■■ models.py
■ ■ ■■■ views.py
■ ■ ■■■ tests.py
■ ■ ■■■ ...
■ ■■■ ...
■■■ frontend/
■ ■■■ src/
■ ■■■ package.json
■ ■■■ ...
■■■ README.md
```

6. How to Run the Project

Backend

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
```

The backend API is available at <http://127.0.0.1:8000/api/disagreements/>.

Frontend

```
cd frontend
npm install
npm run dev
```

The Vite development server normally starts at <http://localhost:5173/>.

7. API

GET /api/disagreements/

Returns the reconciliation disagreements used by the frontend. Supported query parameters include **reason** for filtering and **sort** for value-based sorting.

8. Data Handling

The CSV data contains deliberately dirty values and references. The implementation avoids assuming that every value can be converted directly to an integer. Original values are preserved where necessary so that invalid or non-numeric input does not cause rows to disappear silently.

9. Tenant Isolation

locations.csv is the source of the location-to-organization mapping. Reconciliation results carry the organization associated with the location. The comparison tests include a tenant-boundary case to verify that data from another organization is not exposed.

10. Tests

Tests focus on the comparison logic rather than testing every frontend detail. Covered cases include missing System B records, missing System A records, and tenant isolation. Run the suite with:

```
python manage.py test
```

11. Deliberately Not Built

- Authentication and user login, because the assignment explicitly says to skip authentication.
- Advanced visual design and extensive CSS.
- Performance optimization for very large datasets; the provided dataset is small.

- Production-grade deployment configuration.
- Extra features outside the required reconciliation workflow.

12. How I Worked With the AI Agent

I used an AI coding assistant to help understand the assignment, plan implementation steps, identify bugs, and refine parts of the backend and frontend. I reviewed the generated suggestions and tested the code locally rather than accepting changes blindly. When the implementation produced unexpected behavior, I checked the data, API response, and frontend request flow to identify and correct the issue.

13. README Questions

a. Name one thing the AI agent got wrong. How did you notice?

One issue was related to assumptions about how values from the CSV should be parsed. Some values were not clean numbers, so converting every value directly to an integer could lose or reject data. I noticed this while testing the imported records against the original CSV data and adjusted the handling so the original value is preserved.

b. Which part of your submission are you least confident about, and why?

I am least confident about edge cases involving unusual System B references and duplicate entries because real-world exports can contain more variations than the provided sample. The core cases are covered by tests, but a larger set of production-like examples would provide stronger confidence.

c. If you had a second day, what would you fix first?

I would improve the data-import workflow and add more edge-case tests for malformed references, duplicate records, and mixed value formats. I would also improve the frontend error/loading states while keeping the interface simple.

14. Assignment Scope

The implementation focuses on a small, complete slice of the requested workflow: importing data, reconciling records, enforcing tenant boundaries, exposing results through an API, displaying them in React, filtering and sorting the results, and testing the comparison logic.